LAB TECHNIQUE · CESI

Comparatif d’API Gateways

Kong OSS vs Apache APISIX

POC reproductible, 7 expériences mesurées, une recommandation argumentée

Simon Delcloque

github.com/SkyWace/api-gateway-lab

01 · CONTEXTE

Un point d’entrée unique devant une douzaine de microservices

Contrôle d’accès

Valider les JWT émis par Keycloak

Quotas

10 req/s par client, sans pénaliser les autres

Résilience

Protéger les clients des pannes de backend

Observabilité

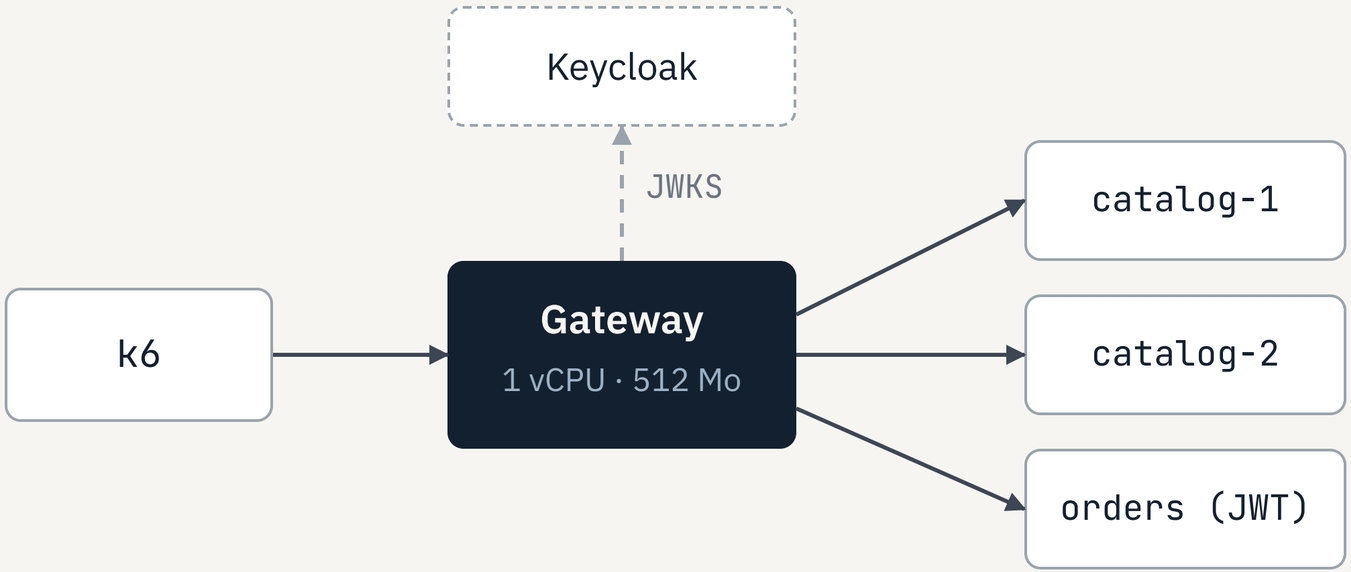
Suivre une requête de bout en bout

Pic visé : 1 500 req/s **Cible :** Kubernetes **Contrainte :** fonctions cœur sans licence payante

Contexte retenu : « Microservices » — entreprise fictive NordFlux (docs/01-cadrage.md)

02 · PROTOCOLE

Le même parcours, les mêmes ressources, pour chaque gateway



Docker Compose · un profil par gateway · k6 dans le réseau Docker

Une gateway à la fois, limites CPU/RAM **vérifiées** par `docker inspect`

Échauffement jeté, puis **médiane de 3 runs**

Une **hypothèse écrite avant** chaque expérience

Journaux bruts versionnés, graphiques générés depuis eux

Deux gateways open source, en édition gratuite

	Kong Gateway OSS 3.9.1	Apache APISIX 3.11
Déploiement POC	DB-less, fichier déclaratif	Standalone, fichier YAML
Validation JWT	Plugin jwt, clé publique copiée	openid-connect, JWKS
Rate limiting	rate-limiting (local)	limit-count (local)
Circuit breaker	Absent (health checks seuls)	api-breaker
Licence	Apache 2.0, lignée OSS gelée en 3.9.x	Apache 2.0, tout inclus

Traefik et Spring Cloud Gateway sont configurés dans le dépôt mais n’ont pas été mesurés (périmètre réduit à deux solutions, comme le sujet l’autorise).

E2 · E7 · SÉCURITÉ

Les deux bloquent tous les tokens invalides

Kong OSS

9 / 9

Clé publique **copiée dans la config** : rotation des clés Keycloak manuelle

Divulgue `Server: kong/3.9.1`

APISIX

9 / 9

Validation par **JWKS** : suit la rotation des clés sans reconfiguration

Messages d'erreur **trop détaillés** (« signature verification failed »)

9 cas : sans token, valide, malformé, signature altérée, payload modifié, exp modifié, alg=none, schéma Basic, route publique · results/*-auth.log

E3 · RATE LIMITING

Même réglage à 10 req/s, précision différente

Client gourmand (30 req/s)

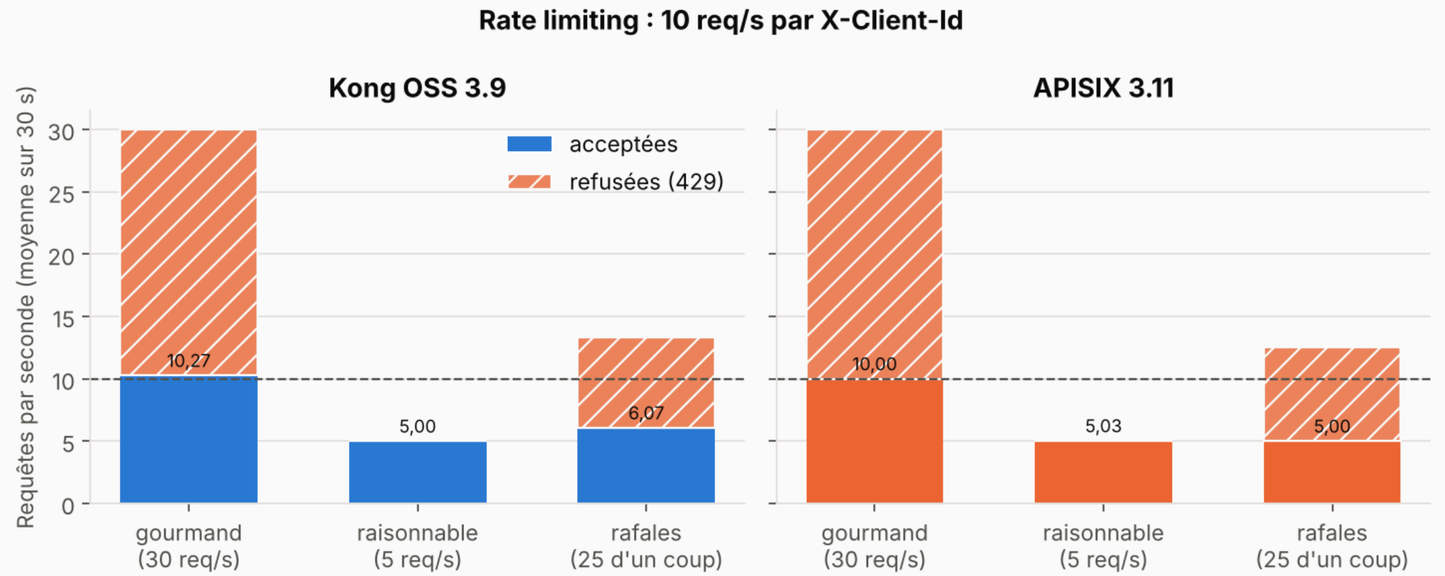
10,27 vs 10,00 req/s

Rafale de 25 requêtes

11,4 vs 10 acceptées

Client raisonnable (5 req/s)

0 refus pour les deux

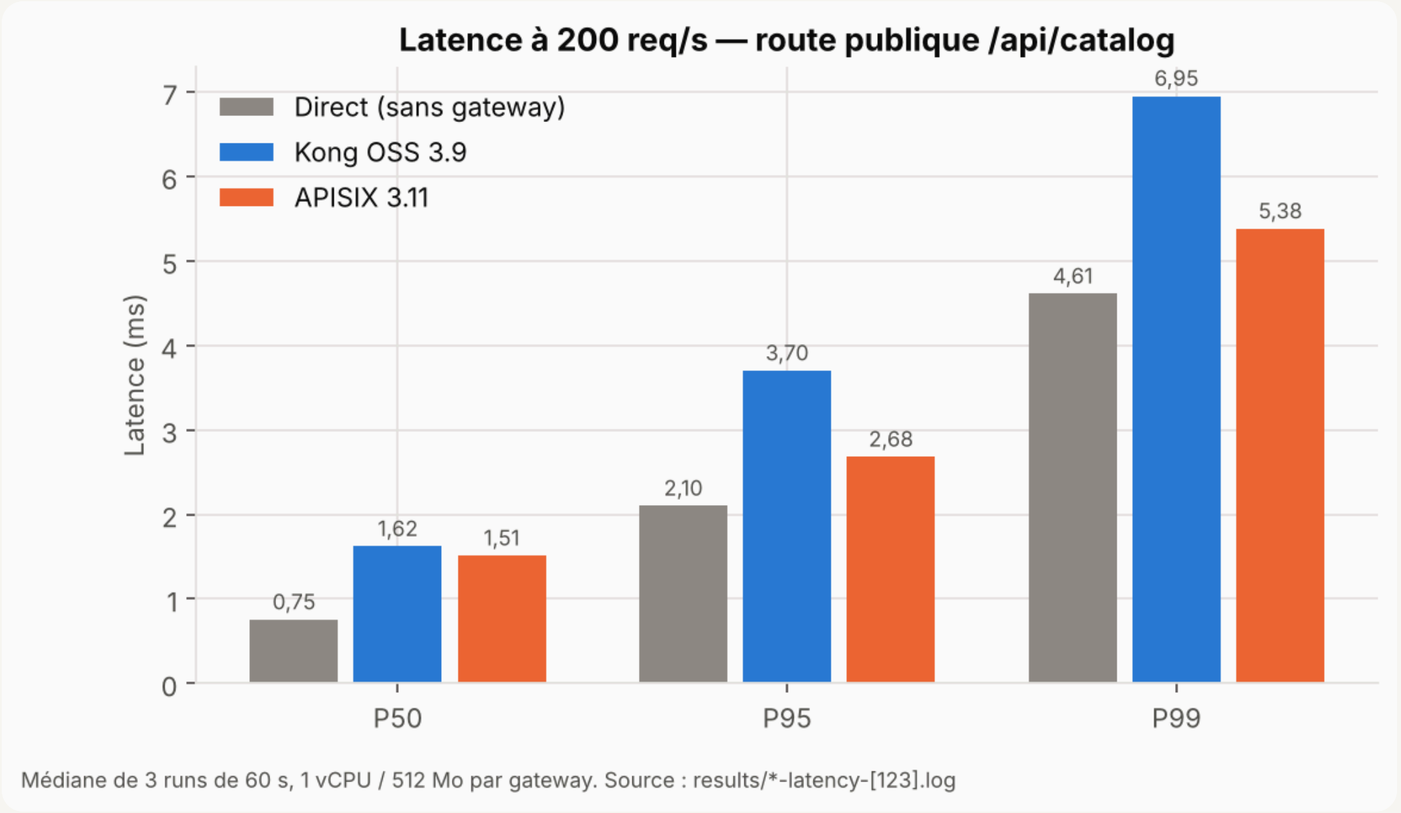


Nombre affiché = débit accepté. Source : results/*-ratelimit.log

Kong en bleu, APISIX en orange · results/*-ratelimit.log

E4 · LATENCE

À 200 req/s, le surcoût reste sous les 2 ms



Surcoût P95

+1,60 ms

+0,58 ms

Coût du JWT (P50)

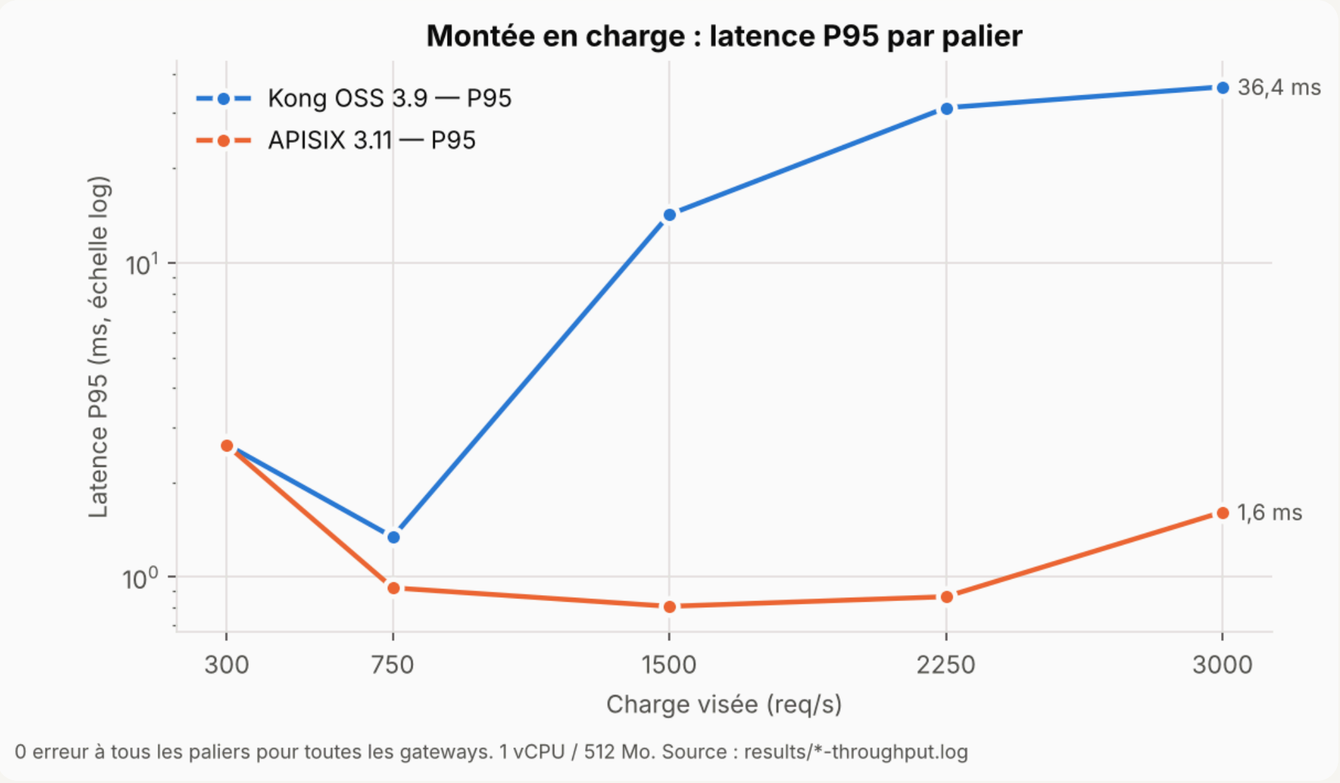
Kong ≈ 0 ms · APISIX +0,5 ms

Le prix du JWKS

Médiane de 3 runs de 60 s, 1 vCPU / 512 Mo · seuil du cadrage : +10 ms · results/*-latency-*.log

E4 · MONTÉE EN CHARGE

À 3 000 req/s, APISIX garde une large marge



0 erreur à tous les paliers pour toutes les gateways. 1 vCPU / 512 Mo. Source : results/*-throughput.log

3 000 req/s	Kong	APISIX
P95	36 ms	**1,6 ms**
CPU	97 %	**53 %**
RAM	205 Mo	**52 Mo**
Erreurs	0	0

Paliers de 30 s, 1 vCPU / 512 Mo, échelle log · Réserve : Kong avait plus de métriques Prometheus actives · results/*-throughput.log

E5 · LE PIÈGE RÉVÉLÉ PAR LES TESTS

Une panne de 30 secondes.

Mal configuré, Kong ne s’en remet jamais.

E5 · RÉSILIENCE

Bien configuré, Kong rejoint APISIX



20 req/s sur /api/orders pendant 120 s. Bande grise en haut = fenêtre de panne (chaos-events.log). Source : results/*-errors-orders.log

Kong, health check passif seul

Cible exclue après 3 erreurs, **jamais réintégrée** : 99,9 % d'erreurs. Erreur de config.

Kong, actif + passif

26,5 % d'erreurs, toutes pendant la panne ; retour à la normale **≈ 5 s** après

APISIX, circuit breaker

24,4 % d'erreurs ; retour **< 5 s**, sans dépendre de /health

50 % de 500 injectés 30 s sur orders, 20 req/s pendant 120 s · perte d'une instance catalogue : 0 erreur pour les deux · results/*-errors-orders*.log

MATRICE DE DÉCISION

APISIX reste devant, avec un écart réduit

Critère	Poids	Kong /5	APISIX /5
Résilience	20 %	4	**5**
Sécurité / contrôle d’accès	20 %	3	**4**
Routage et politiques	15 %	4	**5**
Débit et latence	15 %	3	**5**
Observabilité	15 %	**5**	4
Complexité d’exploitation	10 %	**4**	3
Coûts / licences	5 %	2	**5**
Score pondéré	100 %	**3,70**	**4,45**

Aucun critère éliminatoire déclenché · Pondération « performance d’abord » : 3,57 vs 4,51 · avant correction de la config Kong : 3,30 · docs/03-matrice-decision.md

RECOMMANDATION

Apache APISIX pour NordFlux

Conditions de mise en œuvre

- Réduire les messages d’erreur JWT
- Masquer l’en-tête Server
- Rate limiting sur Redis dès 2 instances
- etcd ou Ingress Controller en production

Kong resterait pertinent si…

- On privilégie l’observabilité sans outillage, à volumes modérés
- On ajoute un health check actif sur chaque upstream
- On accepte la lignée OSS figée en 3.9.x

Aucune solution n’est la meilleure dans l’absolu : cette recommandation vaut pour ce contexte et ces mesures · docs/04-note-decision.md

LIMITES ET SUITE

Ce que les mesures disent, et ce qu’elles ne disent pas

Incidents corrigés

- Limite CPU ignorée par Docker Desktop : détectée, mesures rejouées
- Générateur k6 saturé : 600 VU pré-alloués
- Kong sans health check actif : diagnostiqué, corrigé, rejoué

Validité

- MacBook Air, Docker, sans TLS : résultats en relatif
- Une seule instance de gateway
- Kong avait plus de métriques actives

Prochaines étapes

- Rejouer quotas et résilience sur Kubernetes, 2 instances
- Mesurer le changement de route à chaud
- Rejouer le débit Kong avec métriques allégées

Tout est rejouable : github.com/SkyWace/api-gateway-lab · journal complet dans docs/05-journal-tests.md